

Classification of Business School Admission Status based on Applicant GPA and GMAT Scores

STAT 387: Introduction to Statistical Learning
Winter 2025

Kira Novitchkova-Burbank
Jocelyn Murguia Macias

Rylan Harrison
Andrin Zuellig

Abstract

Thousands of students apply to graduate business programs annually, putting immense stress on admissions officers to review applicant materials and narrow the applicant pool. Statistical learning methods, namely classification models, show potential for automating aspects of the applicant review process by filtering candidates based on academic performance, thus streamlining the selection process. This automation could help reduce the workload for admissions officers, encourage more objective decision-making, and ensure consistency in student selection. We evaluated three classifiers on this task: Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and K-Nearest Neighbors (KNN). From a dataset of 85 observations, each containing an undergraduate GPA, Graduate Management Aptitude Test (GMAT) score, and admission decision (i.e., *admit*, *borderline*, or *do-not-admit*), we created a testing set of 15 applicants by extracting the first 5 applicants from each admission category. The remaining 70 participants (26 *admits*, 23 *do-not-admits*, and 21 *borderlines*) were used to train the classifiers to predict admission decisions. We evaluated the classifiers by computing their associated class-specific sensitivity and specificity, mean sensitivity and specificity, overall error rate, and area under the receiver operating characteristic curve (AUC) on the training and testing data. We also assessed how well the classifiers captured the underlying structure of the data by plotting their decision boundaries. We recommend the KNN classifier for predicting admission decisions given it was the only model to produce an overall error rate of 0 on the testing set, correctly classifying all 15 applicants. The QDA classifier also performed exceptionally on the testing set, producing an overall error rate of 0.067 and AUC of 1. Given the small size of the testing set, we recommend future research adopt cross-validation for more robust evaluation and hyperparameter tuning.

Table of Contents

1	Introduction.....	2
1.1	Statistical Learning Objective.....	2
1.2	Exploratory Data Analysis	3
2	Methodology	7
2.1	Overview	7
2.2	Performance Metrics	7
2.3	Classifiers: Discriminant Analysis	9
2.4	Classifiers: K-Nearest Neighbors	10
3	Results.....	12
3.1	Linear Discriminant Analysis	12
3.2	Quadratic Discriminant Analysis	15
3.3	K-Nearest Neighbors	17
3.4	Performance by Class	20
3.5	Overall Performance	23
4	Conclusion	25
	References.....	27

1 Introduction

1.1 Statistical Learning Objective

Each year, admissions officers review thousands of applications for business school graduate programs. In the 2023-2024 academic year, Harvard Business School alone received 9,856 applications.¹ Manually reviewing applicant materials, whether they be transcripts, personal statements, or resumes, can in turn be an extremely strenuous process, leading to increased stress and burnout. Statistical learning methods – a wide variety of tools for understanding data and the relationships embedded within it – show promising results for automating portions of the applicant review process. In particular, classification algorithms may automate early admission decisions based on academic performance. Three such algorithms, outlined in *An Introduction to Statistical Learning*, are considered in this report: Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and K-Nearest Neighbors (KNN).² The objective is to fit each of these classifiers using a subset of a sample of real applicant data, composed of undergraduate Grade Point Averages (GPAs) and Graduate Management Aptitude Test (GMAT) scores. The trained models will be used to predict applicants' admission statuses for the rest of the data, which can take one of three values: *admit*, *do-not-admit*, or *borderline*. Classifiers will be compared using overall error rates and class-specific performance metrics.

¹ Harvard Business School. (2024). *Class profile: MBA*. <https://www.hbs.edu/mba/admissions/class-profile>

² James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning: With Applications in R*. 2nd Edition, Springer, Berlin. <https://doi.org/10.1007/978-1-0716-1418-1>

1.2 Exploratory Data Analysis

All visualizations in this report were created primarily with the `ggplot2` library.³ The `gridExtra` library was used for combining multiple plots into a single layout (e.g., Figure 2).⁴ We conducted data preprocessing with the `tidyverse` library.⁵

The dataset consists of 85 applicants. To form a test set for model evaluation, the dataset was first grouped by admission status, with the first 5 applicants from each category extracted for testing. The remaining 70 applicants constitute the training set. Every applicant has a recorded GPA, based on a 4-point scale, GMAT result, and admission decision. The GMAT, scored from 205 to 805, tests applicants' skills in quantitative reasoning, verbal reasoning, and data insights, with scores accepted by over 7,700 programs globally.⁶ Among the applicants in the training set, 26 were *admits*, 23 were *do-not-admits*, and 21 were *borderline*. The numerical summary of their GPA and GMAT scores is depicted in Table 1 below.

³ Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>

⁴ Auguie, B. and Antonov, A. (2017). *gridExtra: Miscellaneous Functions for "Grid" Graphics*. R Package Version 2.3. <https://CRAN.R-project.org/package=gridExtra>

⁵ Wickham et al., (2019). Welcome to the Tidyverse. *Journal of Open Source Software*, 4(43), 1686, <https://doi.org/10.21105/joss.01686>

⁶ Graduate Management Admission Council. (2024, January 5). *What is the GMAT exam and what should you know?*. mba.com. <https://www.mba.com/exams-and-exam-prep/gmat-exam/what-is-the-gmat-exam-and-what-should-you-know>

Table 1. Summary Statistics for GPA and GMAT Scores of Training Set Applicants

Minimum	2.130	313.0
1st Quartile	2.638	419.2
Median	3.030	482.5
Mean	2.992	489.9
3rd Quartile	3.365	545.2
Maximum	3.800	693.0

Noticeably, students demonstrate higher academic performance relative to GPA than GMAT scores. The median GPA is 3.030, corresponding to a “B” average on the 4-point scale. In contrast, the median GMAT score was only 482.5, quite low relative to the maximum score of 805. In fact, the highest score any student received was only 693, even though a near-perfect GPA of 3.8 was observed. This suggests strong academic performance in undergraduate coursework does not necessarily correlate to exceptional GMAT scores. Observe also that the middle 50% of applicants scored between 419.2 and 545.2 (as given by the first and third quartile), whereas overall applicants scored between 313 and 693, a much wider range. Hence, GMAT scores are more compressed around the median, with few applicants performing extremely well or poorly. The following scatterplot illustrates the relationship between admission status, GPA, and GMAT scores.

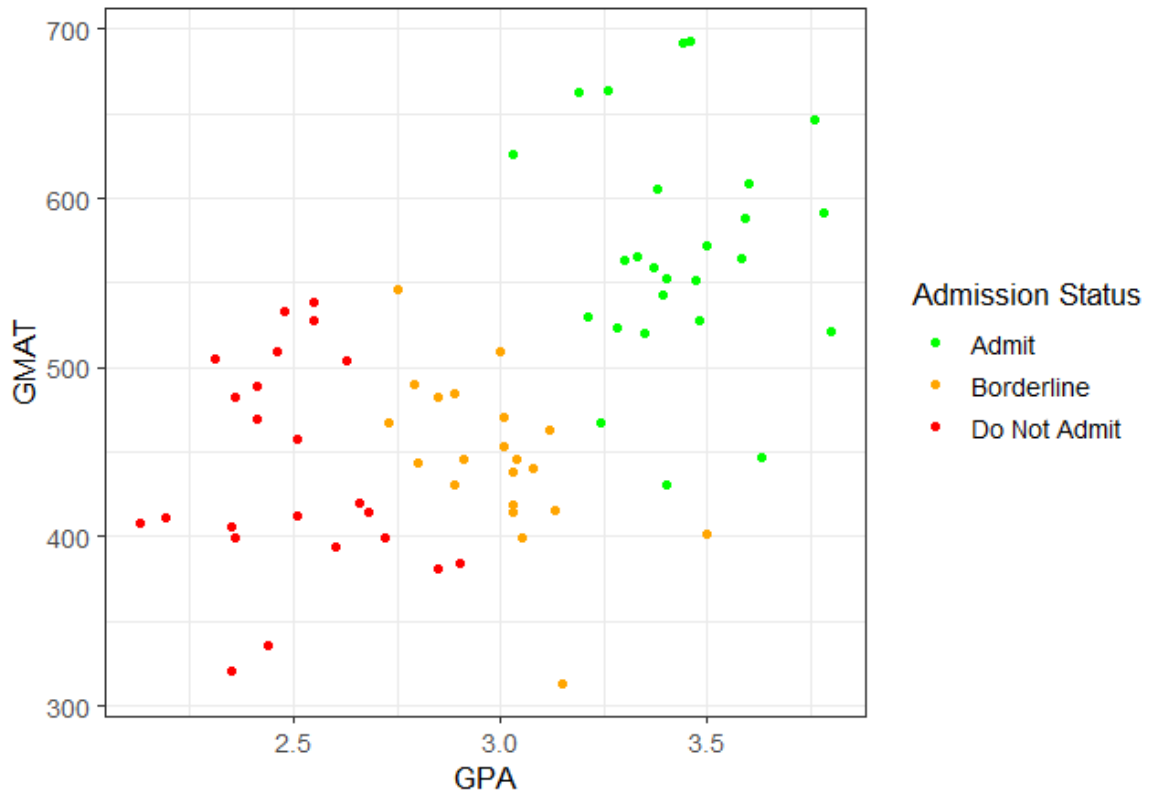


Figure 1. GMAT Scores vs GPA

Figure 1 reflects the positive correlation between GPA and GMAT scores: as GPA increases, GMAT scores rise linearly. The *borderline* and *do-not-admit* students have similar ranges of GMAT scores, while only *admit* students have scores above 600. Hence, while GMAT performance may help identify top applicants, the overlap between *do-not-admit* and *borderline* students suggests low scores are not grounds for rejection alone. In contrast, there is little overlap between the admission categories for GPA. The majority of rejected applicants had GPAs below 2.75. Applicants with GPAs between 2.75 and 3.25 were typically considered *borderline*, while those above 3.25 were usually admitted. Since the applicants appear clustered according to these GPA ranges, GPA is likely to be a better predictor of admission status.

The violin plots in Figure 2 compare the distributions of GMAT scores and GPA across the three admissions groups. For GMAT scores, there is a noticeable overlap between the *borderline* and *do-not-admit* applicants, though the latter skews slightly lower. The *admit* group, in contrast, exhibits higher GMAT scores and less overlap with the other categories. There is a much wider range of GMAT scores for all three groups relative to GPA. This pattern suggests that admission officers do not value GMAT scores as much as GPA when reviewing applications. While a high GMAT score can help top students stand out, it may not be as important for deciding whether they fall into the *borderline* or *do-not-admit* category. In contrast, the distributions of GPAs are more condensed with little overlap between the three groups, reaffirming that GPA more clearly separates *admit*, *borderline*, and *do-not-admit* applicants and thus takes greater precedence for applicant review.

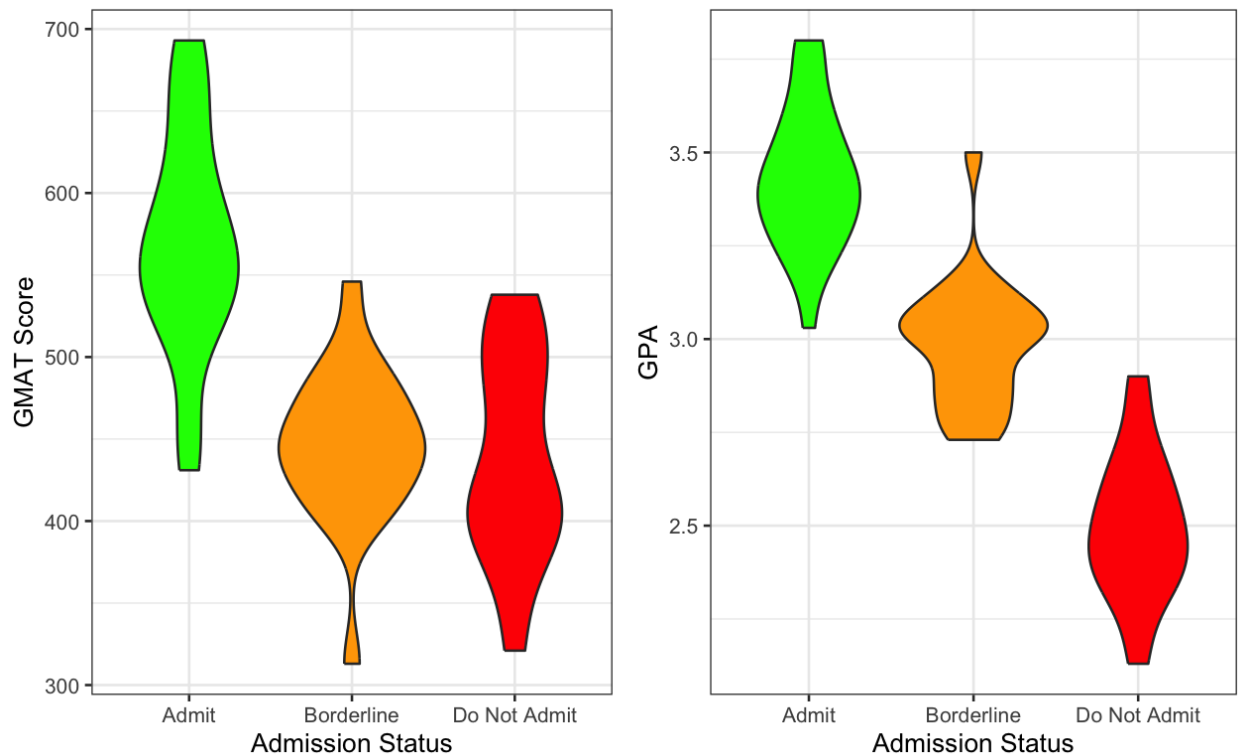


Figure 2. Distribution of GMAT Scores (left panel) and Undergraduate GPA (right panel)

2 Methodology

2.1 Overview

Three classifiers were fit on the data for the 70 participants in the training data using the R programming language: Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and K-Nearest Neighbors (KNN).⁷ Once fit, the remaining 15 participants were used for evaluation. For both the training and testing data, we computed the overall error rate, sensitivity, specificity, mean sensitivity, mean specificity, and the area under the receiver operating characteristic curve (AUC) to compare classifier performance. In addition, we visualized their results through confusion matrices and decision boundaries, modeling our code for the latter after a video tutorial by machine learning professional Yate Ronald.⁸

2.2 Performance Metrics

The first performance metric we computed was the overall error rate, with a lower value indicating better overall model performance, as it represents the proportion of misclassifications. We also considered the class-specific performance metrics of sensitivity and specificity, computed as follows:

$$\text{sensitivity} = \frac{TP}{P}$$

$$\text{specificity} = \frac{TN}{N}.$$

Sensitivity is the proportion of a particular class the model identified correctly. In multiclass problems, each class takes a turn being the “positive” class while all others are considered the

⁷ R Core Team (2023). *R (4.4.1) : A language and environment for statistical computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org>

⁸ Ronald, Y. A. (2024, February 16). *Linear discriminant analysis in R in English*. Spoken Tutorial. <https://spoken-tutorial.org/watch/Machine+Learning+using+R/Linear+Discriminant+Analysis+in+R/English/>

“negative” class. The total number of observed instances of the positive class is denoted P and the number of correct predictions belonging to that class (i.e., the true positives, is denoted TP . Specificity is the proportion of all other observations not misclassified into the positive class. The total number of such observations is denoted as N while TN represents the true negatives: the amount the classifier does not mistakenly classify as the positive class. We computed these metrics using the **caret** package.⁹

We also calculated the related measure of the area under the receiver operating characteristic curve (ROC), commonly abbreviated AUC, using the **pROC** package.¹⁰ In binary classification, the ROC curve plots sensitivity against the false positive rate (the fraction of negative-class observations incorrectly classified as the positive class) for various cutoff values. A cutoff value is the minimum probability of an observation belonging to the positive class needed for it to be classified as the positive class. A perfect classifier would yield a sensitivity of 1 and a false positive rate of 0 for all cutoffs, resulting in an AUC of 1. In contrast, a random classifier would produce the same sensitivity and false positive rate at all threshold values, resulting in an AUC of 0.5. Statisticians David Hand and Robert Till derive that AUC is equivalent to the probability of a randomly selected positive-class observation having a smaller predicted probability of belonging to the negative-class than a randomly selected negative-class observation. They generalize this equality in their definition of multi-class AUC, which is the

⁹ Kuhn, M. (2008). Building Predictive Models in R Using the caret Package. *Journal of Statistical Software*, 28(5), 1–26. <https://doi.org/10.18637/jss.v028.i05>

¹⁰ Robin, X., et al. (2011). Proc: An open-source package for R and S+ to analyze and compare ROC curves. *BMC Bioinformatics*, 12(1). <https://doi.org/10.1186/1471-2105-12-77>

average probability that a randomly selected class i observation has a lower predicted probability of belonging to class j than a randomly selected class j observation for all classes i and j .¹¹

2.3 Classifiers: Discriminant Analysis

Linear and Quadratic Discriminant Analysis are methods that model class-conditional distributions, which are associated with the probability of observing a particular set of predictor variable values given the class of the observation. LDA assumes the conditional probability distribution of a predictor set given an observation belongs to class k is normally distributed with mean vector μ_k and covariance Σ (in practice, this assumption is not verified). It estimates the parameters μ_k and Σ with the training samples. Taking the logarithm of the associated probability density functions and cancelling common terms yields the discriminant functions δ_k , for which these classifiers are named. An observation x is assigned to class k for the k that produces the highest $\delta_k(x)$. The decision boundary between classes i and j is defined as the set of hypothetical observations x such that the associated discriminant functions for i and j are equal: $\delta_i(x) = \delta_j(x)$. The decision boundaries for LDA and QDA are illustrated in Figures 4 through 7. Note that $\delta_k(x)$ is a linear function of x and thus produces linear decision boundaries. QDA works the same way, but it does not assume a constant covariance matrix for each class. The associated discriminant functions and boundaries are in turn quadratic. We implemented LDA and QDA with the **MASS** library for our analysis.¹²

¹¹ Hand, D. J., & Till, R. J. (2001). A Simple Generalisation of the Area Under the ROC Curve for Multiple Class Classification Problems. *Machine Learning*, 45(2), 171–186. <https://doi.org/10.1023/a:1010920819831>

¹² Venables, W. N. & Ripley, B. D. (2002) *Modern Applied Statistics with S* (4th ed.). Springer.

2.4 Classifiers: K-Nearest Neighbors

In KNN, each test data point is assigned to the class most prominent among its K closest neighbors. In particular, we calculate the probability of a test point x_0 belonging to class c as the fraction of points in its set of K nearest neighbors, denoted as $\mathcal{N}_0$, that belong to class c :

$\hat{P}(Y = c|X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = c)$. The c that produces the largest probability is the predicted classification, with ties broken randomly. Since KNN requires identifying every test point's K nearest neighbors with Euclidean distance, we must standardize the predictors so each has a mean of 0 and a standard deviation of 1. Otherwise, predictors of larger magnitudes (in this case, for example, GMAT scores are on a much larger scale than GPAs) will dominate the Euclidean distance calculation. As a result, small-magnitude predictors will unjustifiably have little effect on the predicted classifications. We standardized the training and testing sets separately, after splitting, to prevent characteristics of the test data from seeping into the training.

To choose the optimal K , we fit a KNN classifier 70 times on the training set and computed the associated testing set error, using the `caret` implementation.⁵ For reproducibility, we set a random seed of 70. We initially set K to 1 and incremented by 1 each iteration. The K that minimized test error was 3. In Figure 3, we see that as K increases and the model becomes less flexible, the testing error rate increases dramatically. The error rate also rises for $K < 3$, which suggests overfitting. With $K = 3$, we refit the model and computed the performance metrics described earlier for the training and testing sets.

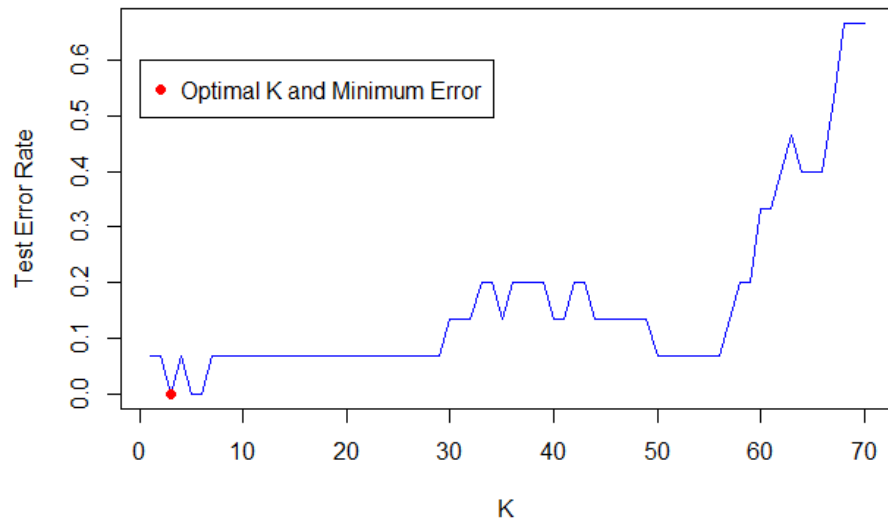


Figure 3. KNN Test Error Rates for $K = 1, 2, \dots, 70$.

3 Results

3.1 Linear Discriminant Analysis

Figure 4 shows the LDA decision boundary plot with training data. The data points correspond to the students' true admission status. The green points represent the 26 students with *admit* status, the orange points represent the 21 students with *borderline* status, and the red points represent the 23 students with *do-not-admit* status. The background colors denote the LDA classifier's predicted classification: green for *admit*, orange for *borderline*, and red for *do-not-admit*. The location of each data point determines its classification. If a data point falls in a region that does not match its true admission status color, it is considered misclassified.

Upon examining Figure 4, we observe that 2 students with *admit* status were misclassified as *borderline*. Additionally, 1 student with *borderline* status was misclassified as an *admit*, while another was misclassified as *do-not-admit*. Finally, 1 student with *do-not-admit* status were misclassified as *borderline*.

These same results of LDA on the training data are displayed in the confusion matrix in Table 2. The confusion matrix shows the LDA-predicted classes as the columns and the true classes as the rows. The diagonal cells represent the correctly predicted classes, while the off-diagonal cells contain the misclassified classes. The confusion matrix indicates the following: 24 students correctly classified as *admit*, 19 students correctly classified as *borderline*, and 21 students correctly classified as *do-not-admit*. The misclassified cases include the 2 students who were classified as *borderline* despite their true status being *admit*. Additionally, 1 student was classified as *admit* and another as *do-not-admit*, while their true status was *borderline*. Finally, 2 students were classified as *borderline* when their true status was *do-not-admit*.

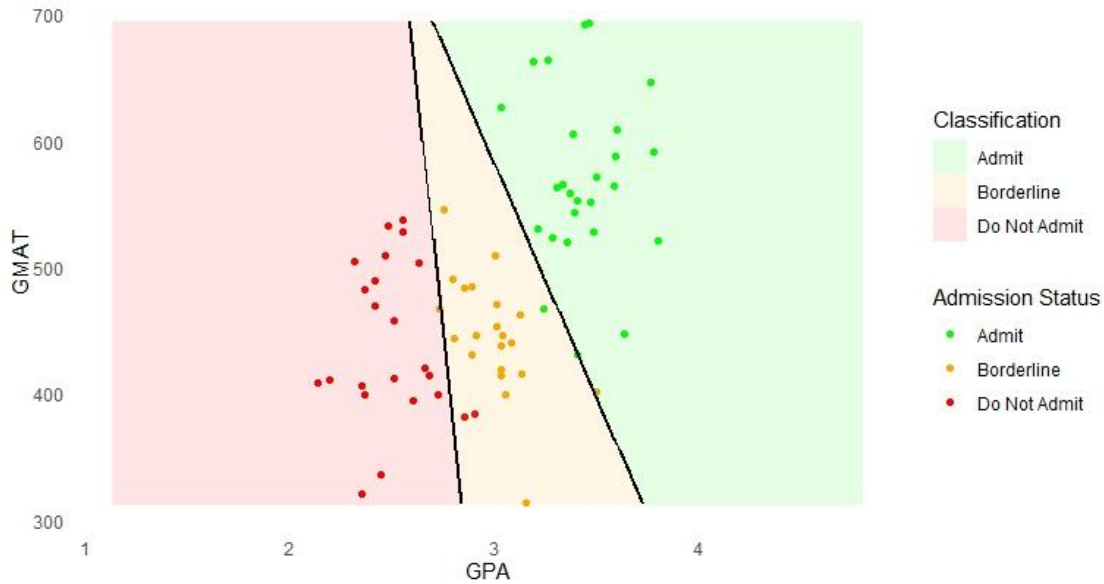


Figure 4. LDA Decision Boundary Plot on Training Data

Table 2. Confusion Matrix of LDA on Training Data

		<i>Predicted Class</i>			Total
		Admit	Borderline	Do Not Admit	
<i>True Class</i>	Admit	24	2	0	26
	Borderline	1	19	1	21
	Do Not Admit	0	2	21	23
	Total	25	23	22	70

Now, we see how the LDA classifier performed on the test data set. Figure 5 displays the decision boundary plot for the same LDA classifier applied to the test data set. The green points represent the 5 students with *admit* status, the orange points represent the 5 students with *borderline* status, and the red points represent the 5 students with *do-not-admit* status.

We observe that 3 students with *admit* status were misclassified as *borderline*. Furthermore, all 5 *do-not-admit* status students were classified correctly, and all 5 *borderline* status students were also classified correctly. Thus, while the LDA classifier effectively distinguishes the *do-not-admit* and *borderline* applicants, it has difficulty separating the *borderline* applicants from those who were admitted. Table 3 breaks down these results in detail.

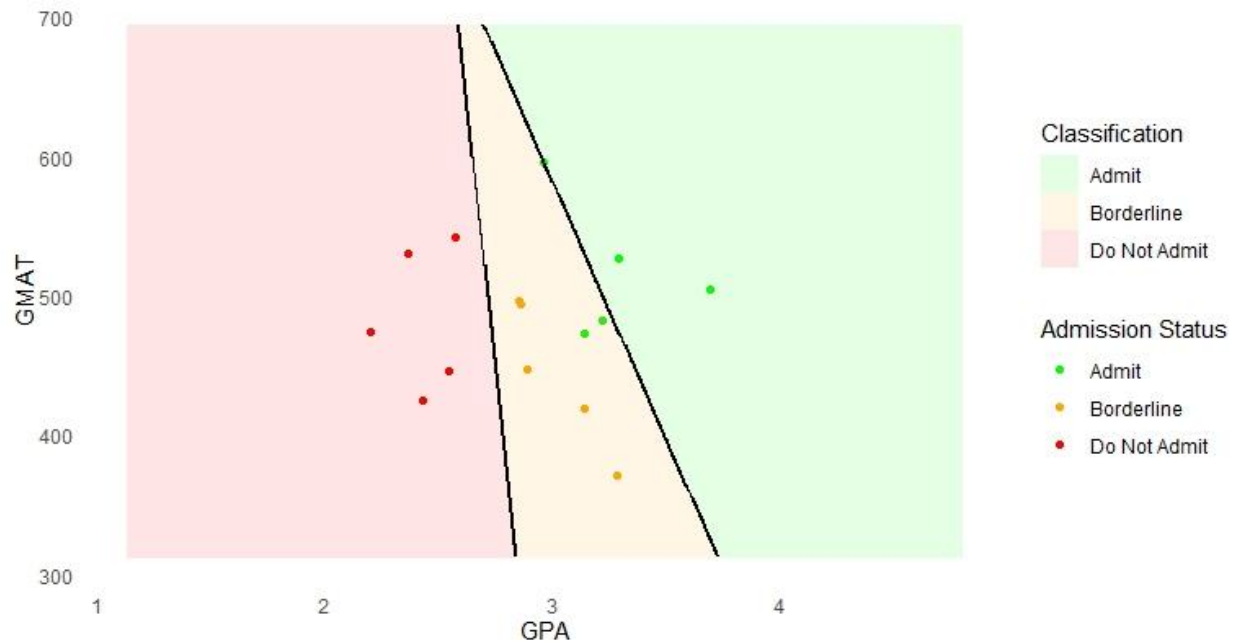


Figure 5. LDA Decision Boundary Plot on Test Data

Table 3. Confusion Matrix of the LDA Classifier on Test Data

		<i>Predicted Class</i>			Total
		Admit	Borderline	Do Not Admit	
<i>True Class</i>	Admit	2	3	0	5
	Borderline	0	5	0	5
	Do Not Admit	0	0	5	5
	Total	2	8	5	15

3.2 Quadratic Discriminant Analysis

Figure 6 shows the QDA decision boundary plot with the training data. We can see that 1 student with *borderline* status was misclassified as *admit* and 1 student with *do-not-admit* status was misclassified as *borderline*. The decision boundaries seem more reasonable than those produced by LDA, as they better capture the slight overlap between the spread of GPA for *do-not-admit* and *borderline* students. Table 4 verifies these findings.

Along the diagonal, the confusion matrix shows 26 students correctly classified as *admit*, 20 students correctly classified as *borderline*, and 22 students correctly classified as *do-not-admit*. The misclassifications in the remaining cells show that 1 student was misclassified as *admit* (whose true classification is *borderline*) and 1 student was misclassified as *borderline* (whose true classification is *do-not-admit*). These misclassifications can be seen in Figure 6 as the 1 orange point within the green region of the plot, and the 1 red point within the orange region of the plot, respectively. Overall, the QDA model has 4 fewer misclassifications than the LDA classifier, suggesting it better captures the structure of the training data.

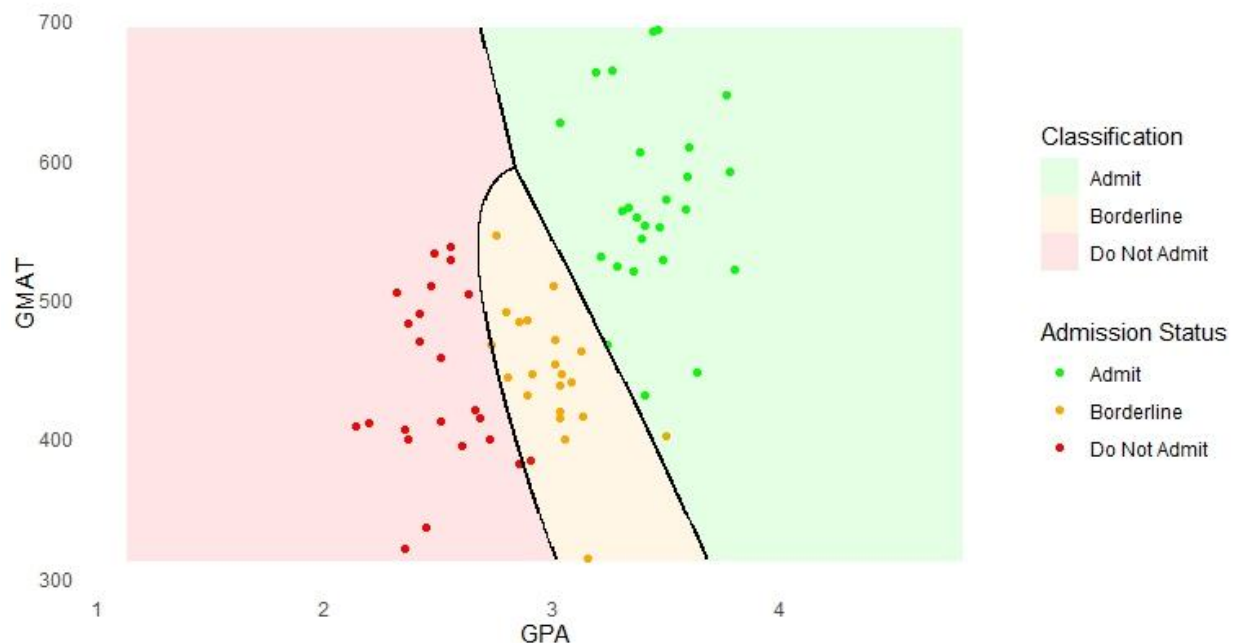


Figure 6. QDA Decision Boundary Plot on Training Data

Table 4. Confusion Matrix of the QDA Classifier on Training Data

		<i>Predicted Class</i>			Total
		Admit	Borderline	Do Not Admit	
<i>True Class</i>	Admit	26	0	0	26
	Borderline	1	20	0	21
	Do Not Admit	0	1	22	23
	Total	27	21	22	70

Following this, the same QDA model was applied to the test data to evaluate how well this classifier fits the unseen data. The decision boundaries are depicted in Figure 7. We can see that only 1 student with *admit* status was misclassified as *borderline*. All *borderline* students and *do-not-admit* status students were correctly classified. Table 5 confirms these results.

In general, QDA does a better job at predicting the students' admission statuses than LDA, producing only 1 misclassification compared to LDA's 3. Like LDA, it also struggled to differentiate the *admit* students from the *borderline* students.

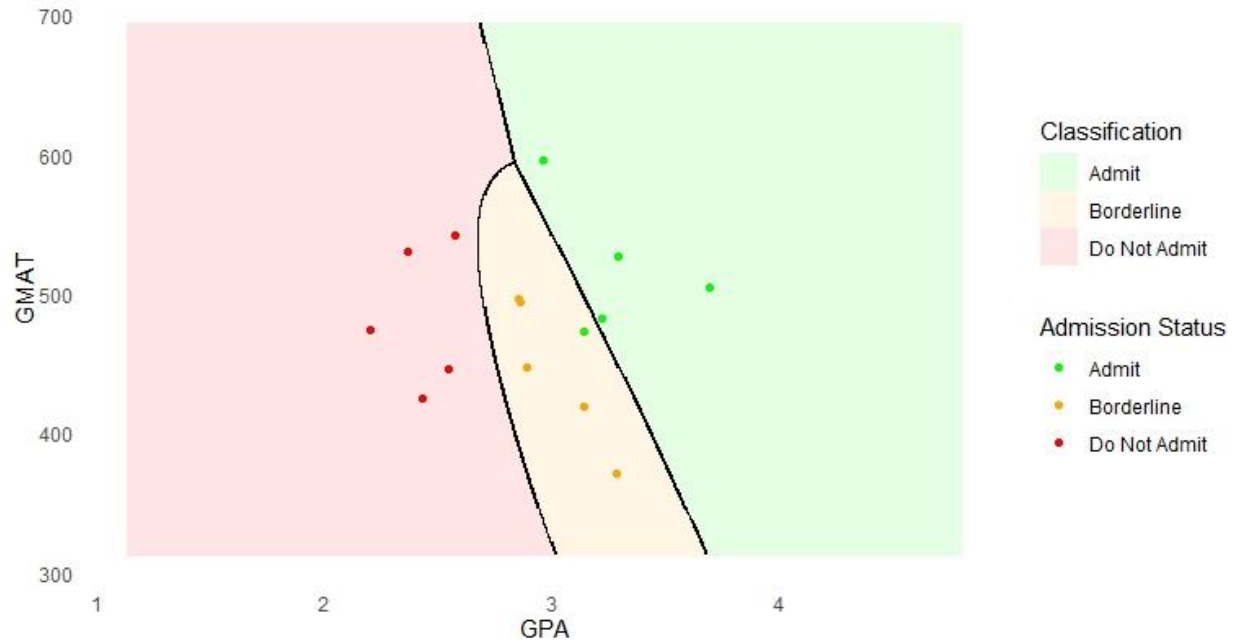


Figure 7. QDA Decision Boundary Plot on Test Data

Table 5. Confusion Matrix of the QDA Classifier on Test Data

		<i>Predicted Class</i>			Total
		Admit	Borderline	Do Not Admit	
<i>True Class</i>	Admit	4	1	0	5
	Borderline	0	5	0	5
	Do Not Admit	0	0	5	5
	Total	4	6	5	15

3.3 K-Nearest Neighbors

Figure 8 shows KNN decision boundaries on the training dataset and Table 6 breaks down the corresponding classification results. We see the KNN model correctly classified 25 *admit* students, 19 *borderline* students, and 23 *do-not-admit* students. The misclassified students include: 1 classified as *borderline* (whose true classification is *admit*) and 2 *borderline* students

who were misclassified into the *do-not-admit* and *admit* categories. These results suggest the KNN model captures the training data well, producing only 1 more misclassification than the QDA model. The decision boundaries are also much more flexible than those produced by LDA and QDA and thus best describe the structure of the data, capturing its nuances. However, like the previous classifiers, it struggles to distinguish between the *borderline* and *admit* statuses.

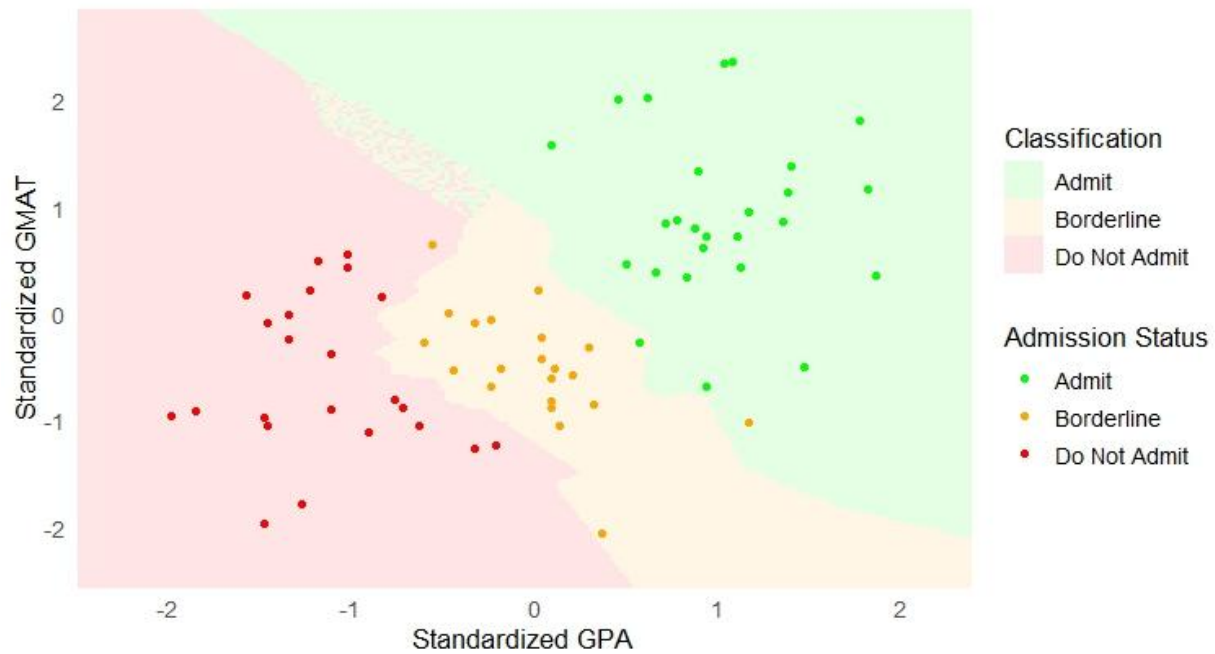


Figure 8. KNN Decision Boundary Plot on Training Data

Table 6. Confusion Matrix of the KNN Classifier on Training Data

		<i>Predicted Class</i>			Total
		Admit	Borderline	Do Not Admit	
<i>True Class</i>	Admit	25	1	0	26
	Borderline	1	19	1	21
	Do Not Admit	0	0	23	23
	Total	26	20	24	70

Figure 9 and Table 7 illustrate how the KNN model classified the applicants in the testing set. We see the KNN model was the only model to correctly classify all 15 applicants in the testing set. While it made one more mistake on the training set than the QDA classifier, its outstanding performance on the testing set suggests it may generalize better to new data.

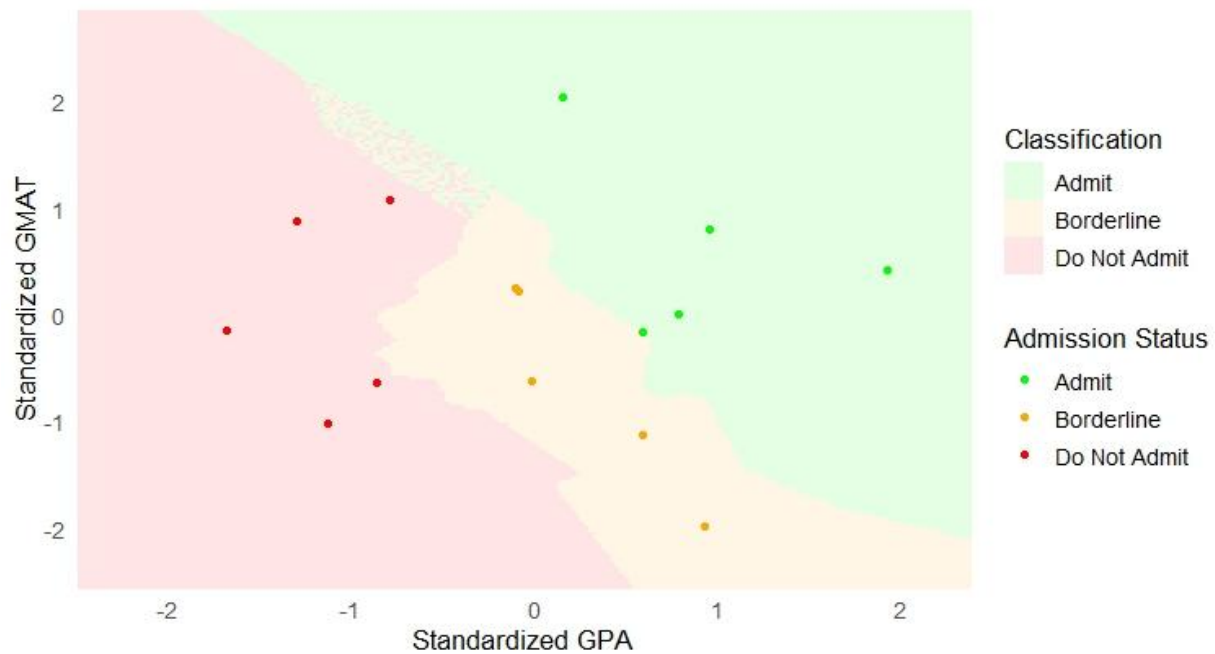
**Figure 9.** KNN Decision Boundary Plot on Test Data

Table 7. Confusion Matrix of the KNN Classifier on Testing Data

		<i>Predicted Class</i>			Total
		Admit	Borderline	Do Not Admit	
<i>True Class</i>	Admit	5	0	0	5
	Borderline	0	5	0	5
	Do Not Admit	0	0	5	5
	Total	5	5	5	15

3.4 Performance by Class

Table 8. Sensitivity and Specificity per Class (i.e. *Admit*, *Borderline*, *Do-Not-Admit*) with Training Data

Classifier	<u>Sensitivity</u>			<u>Specificity</u>		
	Admit	Borderline	Do Not Admit	Admit	Borderline	Do Not Admit
LDA	0.923	0.905	0.913	0.977	0.918	0.979
QDA	1.000	0.952	0.957	0.977	0.980	1.000
KNN	0.962	0.905	1.000	0.977	0.980	0.979

We now proceed to evaluate the results. Table 8 provides a comprehensive summary of the sensitivity and specificity values for each model using training data, broken down into the *admit*, *borderline*, and *do-not-admit* categories. For the *admit* class, the QDA model performed the best with a sensitivity value of 1, followed by KNN with a value of 0.962, and the LDA model having the lowest sensitivity of 0.923. This indicates that the QDA is the best, correctly classifying 100% of the students who should have been admitted into the *admit* category. The LDA and KNN both correctly classified 90.5% of the *borderline* students. In contrast, the QDA model outperformed the others by correctly classifying 95.2% of the *borderline* students. As for

the *do-not-admit* class, each classifier gave different results. The LDA model had the lowest value of 0.931, while the QDA model had a value of 0.957. The KNN model has the highest sensitivity value of exactly 1, correctly classifying all rejected students.

The specificity for each class was also computed for the different classifiers using the training data, as shown also in Table 2. Interestingly, all the classifiers produced identical specificity values of 0.977. This indicates that every model correctly classified 97.7% of the *non-admit* students into classes other than the *admit* category. Furthermore, both the QDA and KNN classifiers have the same highest specificity value of 0.980 for the *borderline* class, meaning they correctly classified 98.0% of the students who do not belong in the *borderline* category into another class. In contrast, the LDA model had the lowest specificity of value 0.918. Although, for the *do-not-admit* class, both the LDA and KNN classifiers had the same specificity value of 0.979. In contrast, the QDA classifier outperformed the others by correctly classifying 100% of the students who are not supposed to be admitted into other categories.

Overall, the QDA classifier had the highest sensitivities for both the *admit* and *borderline* categories, while the KNN classifier produced the highest sensitivity for the *do-not-admit* class. This shows that the QDA model is slightly better at correctly classifying students into their respective categories. In terms of the specificities, QDA performed the best for all three classes, with the exception of the *admit* class, where all three models performed equally well, and the *borderline* class, where both QDA and KNN produced identical results. However, it is important to note that all the sensitivity and specificity values are still very similar, ranging from around 0.9 to 1. Therefore, while the QDA model has the best values for both metrics in most categories, the other models still perform well.

Table 9. Sensitivity and Specificity per Class (i.e. *Admit*, *Borderline*, *Do-Not-Admit*) with Test Data

Classifier	<u>Sensitivity</u>			<u>Specificity</u>		
	Admit	Borderline	Do Not Admit	Admit	Borderline	Do Not Admit
LDA	0.400	1.000	1.000	1.000	0.700	1.000
QDA	0.800	1.000	1.000	1.000	0.900	1.000
KNN	1.000	1.000	1.000	1.000	1.000	1.000

Table 9 provides the class sensitivities and specificities for the test set. Interestingly, for the *admit* category, there was a considerable variation in the sensitivity values: LDA having the lowest sensitivity of 0.4, followed by QDA with a sensitivity of 0.8, and KNN with the highest sensitivity of 1, making it the best classifier for this class. However, for both the *borderline* and *do-not-admit* classes, all three classes produced identical sensitivities. Each classifier accurately classified 100% of the true *borderline* students into the *borderline* category and correctly assigned 100% of the students who should not be admitted into their correct category, as well.

Similar to the sensitivities, all the classifiers had identical specificities for two of the classes. They correctly classified 100% of the students as *admit* or *do-not-admit*. Although, the specificities for the *borderline* class varied, with LDA having the lowest specificity of 0.7, followed by QDA with 0.9, and KNN outperforming the others in this class with a value of 1. This shows that KNN classified all the non-*borderline* students into categories other than *borderline* with 100% accuracy.

In summary, when evaluated with test data, all three models produced equally high sensitivities for both the *borderline* and *do-not-admit* classes. However, only the KNN model had the highest sensitivity for the *admit* category, making it the superior model for accurately

classifying students into their respective categories. As for the specificities, all three models produced identical results for the *admit* class, but KNN again outperformed the other models in the two remaining classes. This suggests that KNN is clearly the best model when considering only sensitivity and specificity. However, the QDA model remains a close second.

3.5 Overall Performance

Table 10. Aggregate Performance Metrics for the Training Data

Classifier	Average Sensitivity	Average Specificity	Overall Error Rate	AUC
LDA	0.914	0.958	0.086	0.993
QDA	0.970	0.986	0.029	0.996
KNN	0.955	0.979	0.043	0.997

Next, we assessed the overall performance of the models. Table 10 shows the overall error rate, AUC, and average sensitivity and specificity computed with the training data. Firstly, we found that the LDA model had the lowest average sensitivity of 0.914, while the KNN model had a value of 0.955. Although, QDA outperformed the others with the highest value of 0.970, correctly classifying 97.0% of students into their respective categories. As for the average specificity of all the classes, the LDA model produced a value of 0.958, making it lowest-performing model in this regard, while both QDA and KNN models had higher values of 0.986 and 0.979, respectively. However, since the QDA model had the highest value, it slightly outperforms the others in classifying students correctly as not belonging to a particular category.

In terms of the overall error rate, the LDA model performed the worst with a value of 0.086. KNN followed a value of 0.043, making it the second worst model. While the QDA classifier produced the lowest overall error rate value of 0.029, meaning it correctly classified 97.1% of the students, and thus emerged as the best model overall. The AUC was also calculated for all the

models. All three models produced values that were quite similar. The LDA model had the lowest AUC at 0.993, followed by QDA with a value of 0.996, and KNN had the highest AUC of 0.997. Although all three models performed well in distinguishing between classes, KNN slightly outperforms the others.

Overall, the QDA classifier outperforms all the models across three of the metrics: average sensitivity, average specificity, and overall error rate. KNN is the only model with an AUC that is one-thousandth higher than that of QDA. Therefore, given the overall error rate's importance in determining the optimal model and its top performance across most metrics, the QDA model most effectively captures the patterns in the training data, followed by KNN. However, it is important to note that the LDA model has only slightly lower values, and thus also performs well.

Table 11. Aggregate Performance Metrics for the Test Data

Classifier	Average Sensitivity	Average Specificity	Overall Error Rate	AUC
LDA	0.800	0.900	0.200	0.980
QDA	0.933	0.967	0.067	1.000
KNN	1.000	1.000	0.000	1.000

Lastly, the average sensitivity of all the classes, average specificity of all the classes, overall error rate, and AUC were found for the classifiers with test data, with the results displayed in Table 11. As seen in this table, the average sensitivity was 0.800 for the LDA model, 0.933 for the QDA model, and 1 for the KNN model. Since KNN had the highest value, it does a better job than the other models in correctly classifying students across all categories on average. As for the average specificity across all the classes, the LDA, QDA, and KNN classifiers produced values of 0.900, 96.7, and 1.00, respectively. Because KNN perfectly classified 100% of students as not belonging to specific class, it again outperformed both the other models.

In addition, the overall error rate was calculated for the models using the test data, also presented in Table 11. The overall error rate value for LDA was calculated to be 0.2, for QDA it was 0.067, and for KNN it was 0. Therefore, the KNN classifier was the only model to correctly classify 100% of the test data. Lastly, the AUC was determined. With all the models producing AUC values close to or equal to 1, it shows that all the classifiers are really effective at distinguishing between classes. However, the LDA model has slightly lower AUC value of 0.980, while both the QDA and KNN models had a perfect value of 1, demonstrating their exceptional ability at differentiating between classes.

4 Conclusion

On the testing set, the KNN classifier demonstrates perfect performance across all metrics: average sensitivity, average specificity, overall error rate, and AUC. However, it is important to note that the K value for the KNN model was selected based on the testing error rates. Thus, while the K value was optimal for this particular test data, it may not generalize well to new unseen applicants; the KNN classifier may be overfit. Another viable option for choosing K is k -fold cross validation, where the training set is divided into k subsets and the KNN classifier is fit k times on $k - 1$ of those subsets. The remaining subset is used for evaluation, enabling approximation of the testing set error rate and hyperparameter tuning without exposure to the testing set designated for assessing the final model's performance. We urge future studies to implement cross-validation and compare their results with those presented here.

The QDA model also performed exceptionally on the testing data, only misclassifying one applicant and yielding a perfect AUC value of 1. This result implies that even when the QDA model misclassified applicants, the associated predicted probability was lower than those produced for correct classifications, suggesting it can still confidently distinguish between the

different classes. While the KNN model is recommended to achieve the optimal results, QDA is a close second. Based on our exploratory data analysis, GMAT scores may not be necessary for producing accurate decision predictions. Future studies should consider fitting these models without this predictor to determine if it is possible to construct a simpler, robust classifier.

References

1. Harvard Business School. (2024). *Class profile: MBA*.
<https://www.hbs.edu/mba/admissions/class-profile>
2. James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning: With Applications in R*. 2nd Edition, Springer, Berlin.
<https://doi.org/10.1007/978-1-0716-1418-1>
3. Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>
4. Auguie, B. and Antonov, A. (2017). *gridExtra: Miscellaneous Functions for “Grid” Graphics*. R Package Version 2.3. <https://CRAN.R-project.org/package=gridExtra>
5. Wickham et al., (2019). Welcome to the Tidyverse. *Journal of Open Source Software*, 4(43), 1686, <https://doi.org/10.21105/joss.01686>
6. Graduate Management Admission Council. (2024, January 5). *What is the GMAT exam and what should you know?*. mba.com. <https://www.mba.com/exams-and-exam-prep/gmat-exam/what-is-the-gmat-exam-and-what-should-you-know>
7. R Core Team (2023). *R (4.4.1) : A language and environment for statistical computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org>
8. Ronald, Y. A. (2024, February 16). *Linear discriminant analysis in R in English*. Spoken Tutorial. <https://spokentutorial.org/watch/Machine+Learning+using+R/Linear+Discriminant+Analysis+in+R/English/>
9. Kuhn, M. (2008). Building Predictive Models in R Using the caret Package. *Journal of Statistical Software*, 28(5), 1–26. <https://doi.org/10.18637/jss.v028.i05>

10. Robin, X., Turck, N., Hainard, A., Tiberti, N., Lisacek, F., Sanchez, J.-C., & Müller, M. (2011). Proc: An open-source package for R and S+ to analyze and compare ROC curves. *BMC Bioinformatics*, 12(1). <https://doi.org/10.1186/1471-2105-12-77>
11. Hand, D. J., & Till, R. J. (2001). A Simple Generalisation of the Area Under the ROC Curve for Multiple Class Classification Problems. *Machine Learning*, 45(2), 171–186. <https://doi.org/10.1023/a:1010920819831>
12. Venables, W. N., & Ripley, B. D. (2002). *Modern applied statistics with S* (4th ed.). Springer.